Year: 2026-2027 Sem V-NEP

Course: Agile Software Development & DevOps Lab

Class: T1 Date of Submission: 06/10/2026

A Mini Project Report on

FoodExpress

An Online Food Ordering System Delivered Through an Agile and DevOps Workflow

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Nishant Fulwani	2401026
Mohit Hotwani	2401037
Kyra Kakar	2401038

Under the guidance of

Prof. ____Sanober Sheikh_____

Department of ____AI & DS_____

_____Thadomal_____ College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “FoodExpress: An Online Food Ordering System Delivered Through an Agile and DevOps Workflow” is a bona fide work of Nishant Fulwani (2401026), Mohit Hotwani (2401037) and Kyra Kakar (2401038), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____ _____

Project Guide **Head of Department**

Date: _____

Place: _____

ABSTRACT

FoodExpress is an Online Food Ordering System developed to demonstrate how the concepts learned in the ASD & DevOps laboratory can be integrated into a single working software project. The application was developed using HTML, CSS, Bootstrap, and JavaScript for the frontend, Python Flask for the backend, and MySQL as the database in the containerized environment. Customers can register and log in, browse available food items, add items to their cart, modify quantities, place orders, and view their order history and status, while the administrator can manage food items, view customer orders, and update order statuses.

The project progressively applies the seven laboratory experiments. Git and GitHub provide source-code and version management; Docker packages the Flask application into a portable container; Docker Compose runs Flask and MySQL as interconnected services; Jenkins provides Continuous Integration and is extended to automate Docker image building and deployment; Jira demonstrates an Agile Scrum workflow; and Prometheus provides application monitoring through the /metrics endpoint. This produces a unified software-development and DevOps workflow rather than isolated demonstrations of individual tools.

Keywords: Agile, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Flask, MySQL

CONTENTS

Chapter 1 Introduction

Chapter 2 Requirements and Agile Planning (Experiment 6)

Chapter 3 System Design

Chapter 4 Application Implementation

Chapter 5 DevOps Implementation (Experiments 1–7)

Chapter 6 Testing

Chapter 7 Implementation

Chapter 8 Results and Discussion

Chapter 9 Conclusion and Future Scope

A Appendix: Running the Project

1. Introduction

1.1 Background

FoodExpress is a web-based Online Food Ordering System developed as the mini project for the ASD & DevOps laboratory. The project provides a practical context in which the concepts taught through the laboratory experiments can be applied progressively to one application.

The laboratory covers version control, containerization, multi-service deployment, continuous integration, automated deployment, Agile project management and continuous monitoring. FoodExpress brings these practices together so that development, deployment, planning and monitoring form one connected lifecycle.

1.2 Problem Statement

To design and develop a web-based Online Food Ordering System and deliver it through an integrated Agile and DevOps workflow in which source code is version-controlled, the application is containerized, dependent services are orchestrated, changes are automatically tested and deployed, development tasks are tracked through Jira, and the running application is monitored.

1.3 Objectives

1. Develop an online food ordering application with customer and administrator functionality.

2. Allow customers to register, log in, browse food items, manage a cart, place orders and view order history/status.

3. Allow the administrator to manage food items, view customer orders and update order statuses.

4. Use Git and GitHub for source-code version control, branching, commits and remote collaboration.

5. Containerize the Flask application using Docker and run Flask and MySQL as interconnected services using Docker Compose.

6. Configure Jenkins for Continuous Integration and extend the pipeline for automated Docker image building and deployment.

7. Use Jira to represent an Agile Scrum workflow for planning and tracking development tasks.

8. Integrate Prometheus to collect and monitor application metrics.

1.4 Scope

The project scope is the FoodExpress web application and the DevOps workflow demonstrated through Experiments 1 to 7. The source report focuses on the customer and administrator application functions and on integrating Git, GitHub, Docker, Docker Compose, Jenkins, Jira and Prometheus.

1.5 Mapping of Lab Experiments to the Project

Exp.	Lab Topic	Application in FoodExpress
1	Version Control using Git and GitHub	Repository, commits, branches, remote repository and merging
2	Containerization using Docker	Dockerfile, image, container and port mapping
3	Multi-Service Deployment using Docker Compose	Flask + MySQL services, volume, health check and service networking
4	Continuous Integration using Jenkins	Jenkins pipeline, pytest, GitHub webhook
5	Automated Deployment using GitHub and Jenkins	Docker image build and Docker Compose deployment
6	Agile Software Development Workflow using Jira	Scrum-style task tracking and workflow
7	Continuous Monitoring using Prometheus	Flask metrics, /metrics endpoint and PromQL monitoring

Table 1.1: Lab experiments and their application in FoodExpress

2. Requirements and Agile Planning

2.1 User Roles

Role	Main Functionality
Customer	Register/login, browse food items, manage cart, place orders and view order history/status
Administrator	Manage food items, view customer orders and update order statuses

Table 2.1: User roles

2.2 Functional Requirements

• Customers can register and log in to the system.

• Customers can browse available food items.

• Customers can add food items to a cart and modify quantities.

• Customers can place food orders.

• Customers can view their order history and current order status.

• The administrator can manage food items.

• The administrator can view customer orders.

• The administrator can update order statuses.

2.3 Non-Functional / DevOps Requirements

• The application should run in a consistent environment through Docker.

• Flask and MySQL should communicate as interconnected services through Docker Compose.

• Jenkins should automate source retrieval, dependency setup and testing, with deployment added to the extended pipeline.

• Jira should provide an Agile/Scrum workflow for development tasks.

• Prometheus should collect application metrics exposed by the Flask application.

2.4 Scrum-Style Workflow

FoodExpress used Jira as the Agile planning and tracking tool. A Scrum-style workflow was followed in which a practical feature task, “Add About section to FoodExpress homepage,” was moved through the following stages:

To Do
↓
In Progress
↓
In Review
↓
Done

The feature was developed using the Git/GitHub workflow, tested, pushed to the repository, reviewed and then verified through Jenkins before the Jira issue was moved to Done.

2.5 Agile-to-DevOps Traceability

Stage	Tool / Practice	Outcome
Planning	Jira	Task created and tracked through Agile stages
Development	Git / GitHub	Source code versioned and changes pushed
Verification	Jenkins	Automated setup and tests executed
Deployment	Docker / Docker Compose	Application deployed in containers
Completion	Jira	Task moved to Done after successful verification

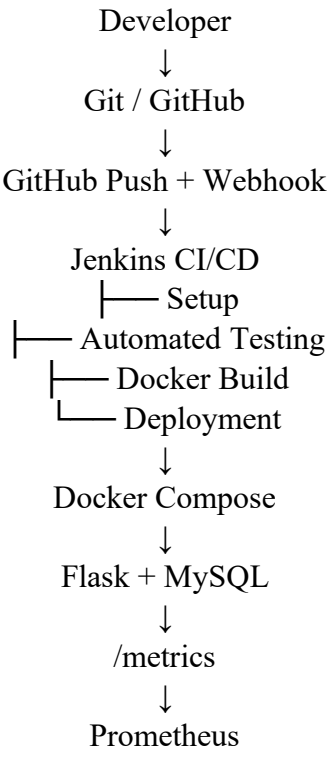
Table 2.2: Agile and DevOps workflow traceability

3. System Design

3.1 Architecture

FoodExpress consists of a frontend, a Python Flask backend and a MySQL database. The application and database are deployed in containers. Jenkins manages the automated delivery workflow, Jira manages Agile planning, and Prometheus monitors the running application.

The overall integrated workflow described in the original project report is:



Jira → Agile planning and task tracking

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, Bootstrap, JavaScript	Provides the web interface for customers and administrators
Backend	Python Flask	Implements the application logic and web services
Database	MySQL	Stores application data including food items and orders
Version Control	Git and GitHub	Tracks changes, branches, commits and remote source code
Containerization	Docker	Packages the Flask application and dependencies
Orchestration	Docker Compose	Runs Flask and MySQL as interconnected services
CI/CD	Jenkins	Automates setup, testing, Docker build and deployment

Agile Management	Jira	Plans and tracks development tasks
Monitoring	Prometheus	Collects application metrics from /metrics

Table 3.1: Technology stack

3.3 Service Design

Service	**Responsibility**
Flask web service	Runs the FoodExpress application and exposes application endpoints and /metrics
MySQL database service	Stores FoodExpress data and provides persistent database functionality
Prometheus service	Scrapes metrics from the Flask application and supports PromQL analysis

Table 3.2: Service design

3.4 Monitoring Design

The prometheus-flask-exporter package was integrated with Flask to expose application monitoring information through the /metrics endpoint. Prometheus was added to the Docker Compose environment and accessed the Flask service through the Compose network.

The Prometheus target page was used to verify that the FoodExpress application was UP. PromQL queries were used to analyse HTTP request counts according to method and response status, while request-duration metrics allowed monitoring beyond a simple container availability check.

4. Application Implementation

4.1 Application Features

The FoodExpress application provides separate functionality for customers and administrators. Customers can register and log in, browse available food items, add items to their cart, modify quantities, place orders, and view order history and status. Administrators can manage food items, view customer orders and update order statuses.

4.2 Customer Workflow

The customer-side workflow begins with registration/login, followed by browsing food items and managing the cart. The customer then places an order and can subsequently view the order history and status.

Register / Login
↓
Browse Food Items
↓
Add to Cart

↓
Modify Quantity
↓
Place Order
↓
View Order History / Status

4.3 Administrator Workflow

The administrator can manage the food catalogue, view customer orders and update order statuses. This provides a separate administrative workflow from the customer ordering process.

Manage Food Items
↓
View Customer Orders
↓
Update Order Status

4.4 Project Repository and Application Structure

The Git repository maintained the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests and monitoring configuration. The source report shows that these project files were committed and pushed to the GitHub repository as part of Experiment 1.

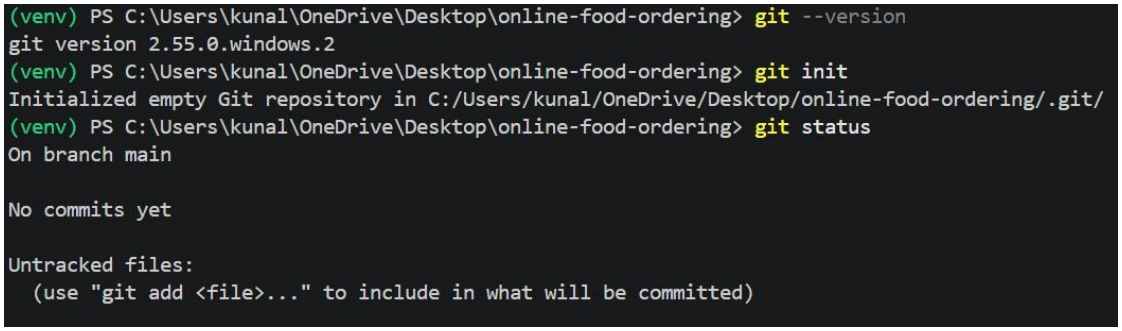


Figure 4.1: FoodExpress project repository / version-control evidence



Figure 4.2: FoodExpress GitHub repository evidence

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git commit -m "Initial FoodExpress application"
[main (root-commit) eabb3c9] Initial FoodExpress application
 26 files changed, 2647 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 Dockerfile
 create mode 100644 Jenkinsfile
 create mode 100644 README.md
 create mode 100644 app.py
 create mode 100644 app/__init__.py
 create mode 100644 app/models.py
 create mode 100644 app/routes.py
 create mode 100644 app/static/css/style.css
 create mode 100644 app/static/js/script.js
```

5.

DevOps Implementation

5.1 Version Control Using Git and GitHub

Git was used as the distributed version-control system for FoodExpress. The project repository maintained the Flask backend, HTML templates, static files, Docker configuration, Jenkinsfile, tests and monitoring configuration.

Changes were staged and committed using Git, while GitHub was used as the remote repository. The local repository was connected using git remote add origin and project changes were pushed to GitHub. git log --oneline was used to inspect commit history, while branches were used to isolate feature development before merging changes into the main branch.

LO2: “Use Git effectively for version control, source code management and collaborative team management.”

Figure 5.1: Git initialization and repository status

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git log --oneline
eabb3c9 (HEAD -> main) Initial FoodExpress application
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git branch
* main
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git status
On branch main
nothing to commit, working tree clean
```

Figure 5.2: Git configuration and initial commit

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git remote add origin https://github.com/MohitHotwani10/o
nline-food-ordering.git
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git remote -v
origin  https://github.com/MohitHotwani10/online-food-ordering.git (fetch)
origin  https://github.com/MohitHotwani10/online-food-ordering.git (push)
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git push -u origin main
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 4 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (24/24), 12.10 KiB | 774.00 KiB/s, done.
Total 24 (delta 7), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (7/7), done.
To https://github.com/MohitHotwani10/online-food-ordering.git
 * [new branch]      main -> main
```

Figure 5.3: Git log and branch information

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git checkout -b feature/update-readme
Switched to a new branch 'feature/update-readme'
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git branch
* feature/update-readme
  main
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git status
On branch feature/update-readme
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git add README.md
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git commit -m "Update project README"
```

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git log --oneline --all --graph
* a7db560 (HEAD -> feature/update-readme) Update project README
* eabb3c9 (origin/main, main) Initial FoodExpress application
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git merge feature/update-readme
Updating eabb3c9..a7db560
Fast-forward
 README.md | 43 +++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 43 insertions(+)
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
```

Figure 5.4: Git feature branch and merge operation

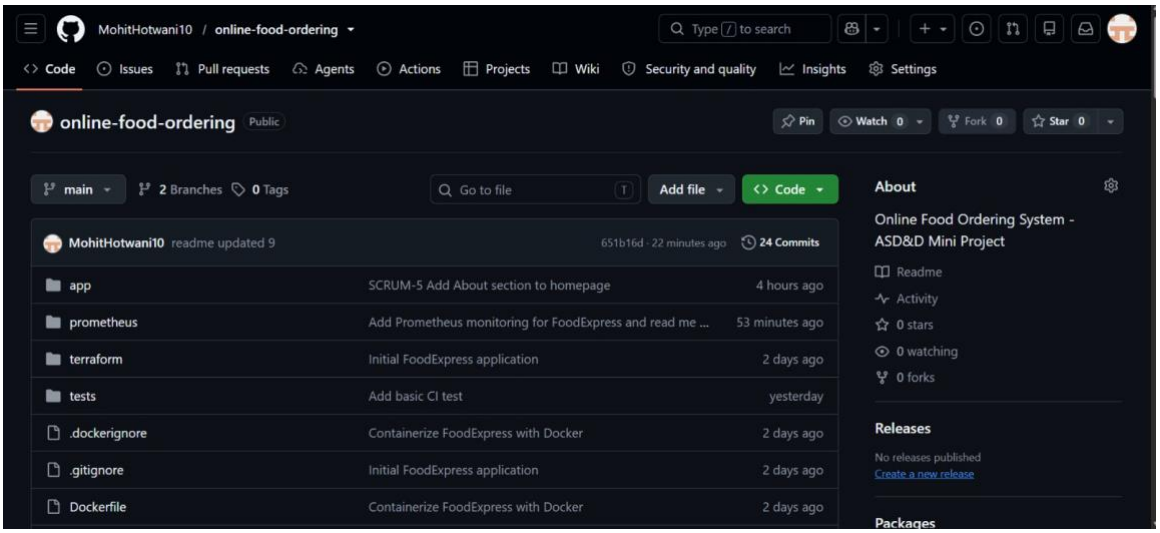


Figure 5.5: GitHub repository and pushed project changes

Figure 5.6: Remote repository and push operation

5.2 Containerization Using Docker

Docker packages an application together with its dependencies, libraries, runtime and configuration into a consistent environment. A Docker image acts as the template, while a container is a running instance created from that image.

For FoodExpress, a Dockerfile was created using a Python base image. It established the application working directory, copied requirements.txt, installed the required packages, copied the source code, exposed port 5000 and started the Flask application.

The image was built using:

docker build -t foodexpress .

A container was created from the image and port 5000 of the container was mapped to port 5000 of the host. docker images, docker ps, docker logs, docker stop and docker start were used for container management and troubleshooting.

LO4: “Containerisation, orchestrate and deploy applications using Docker, Compose, Kubernetes and cloud infrastructure tools.”

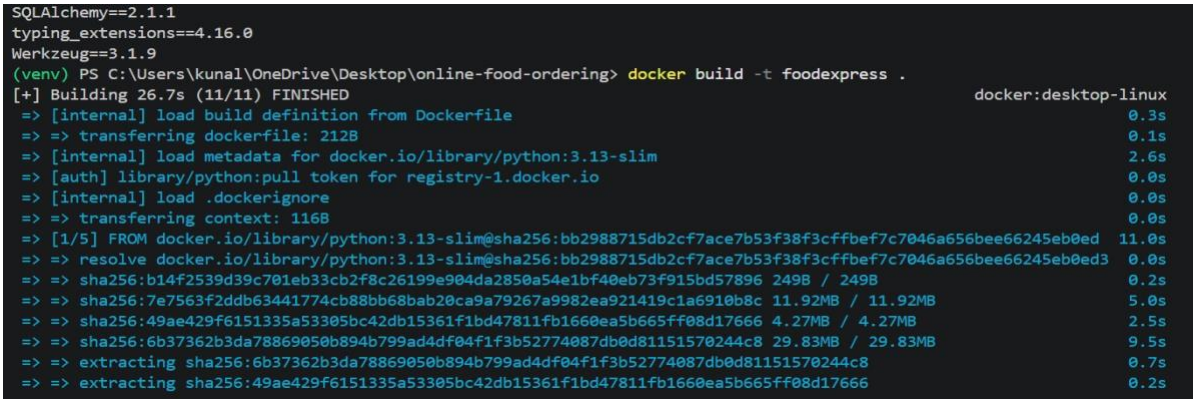


Figure 5.7: Docker image/build evidence

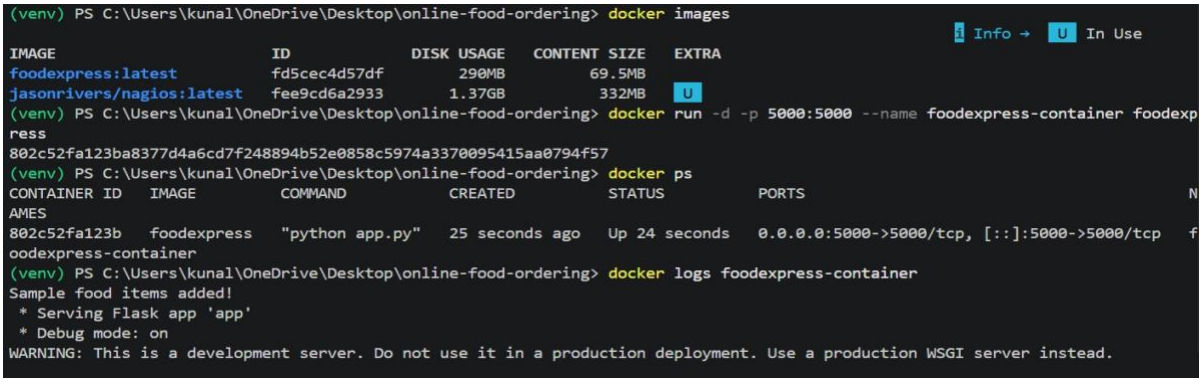


Figure 5.8: Docker container and logs

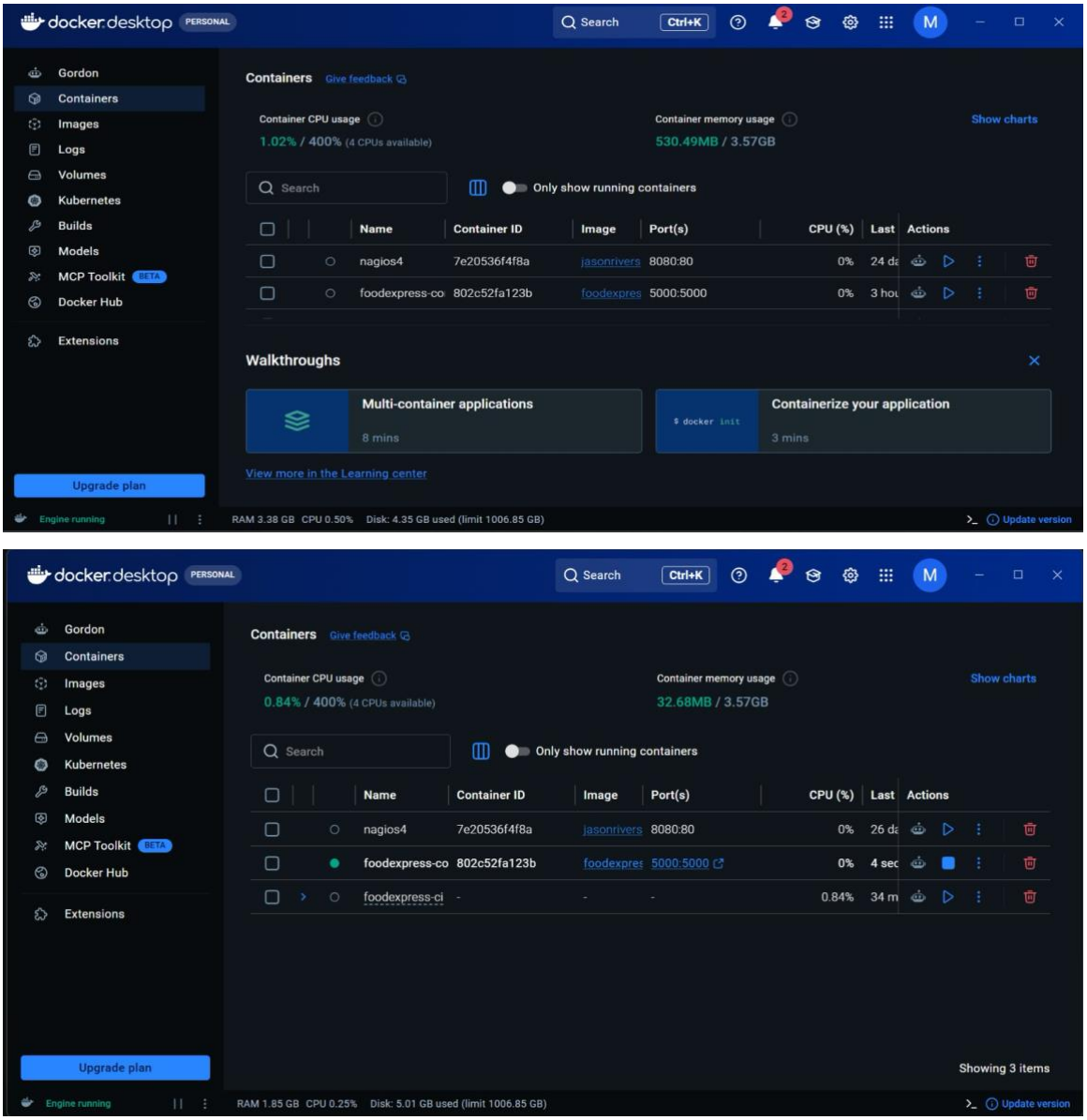


Figure 5.9: Docker Desktop container and image status

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> docker start foodexpress-container
foodexpress-container
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> docker ps
CONTAINER ID   IMAGE          COMMAND           CREATED        STATUS          PORTS                                       NAMES
802c52fa123b   fd5cec4d57df   "python app.py"   45 hours ago   Up 28 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   foodexpress-co
ntainer
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering>
```

Figure 5.10: FoodExpress Docker container execution

Figure 5.11: Docker Desktop container view

5.3 Multi-Service Deployment Using Docker Compose

Docker Compose extends Docker to multi-service applications by allowing related services to be declared in a single YAML configuration file. Each service can run in its own container while communicating with the other services.

FoodExpress requires both an application server and a database, so docker-compose.yml was created. The web service runs the Flask application and the db service uses a MySQL image. The Flask container communicates with MySQL using the Compose service name db and the DATABASE_URL environment variable.

A named Docker volume, mysql_data, provides persistent database storage, while a MySQL health check ensures that the Flask service waits for the database service to become healthy before starting. Commands such as docker compose config, docker compose up -d, docker compose ps and docker compose logs were used for validation and troubleshooting.

LO4: “Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.”

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> docker compose config
name: online-food-ordering
services:
  db:
    container_name: foodexpress-db
    environment:
      MYSQL_DATABASE: foodexpress
      MYSQL_PASSWORD: foodpass
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_USER: fooduser
    healthcheck:
      test:
        - CMD
```

Figure 5.12: Docker Compose configuration

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> docker compose up --build -d
[+] up 13/13
 ✓ Image mysql:8.4 Pulled
[+] Building 4.9s (13/13) FINISHED
 => [internal] load local bake definitions
 => => reading from stdin 581B
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 212B
 => [internal] load metadata for docker.io/library/python:3.13-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 116B
```

Figure 5.13: Docker Compose build and startup

```
(venv) PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> docker compose ps
NAME             IMAGE                     COMMAND                  SERVICE   CREATED          STATUS                    PORTS
foodexpress-db   mysql:8.4                 "docker-entrypoint.s…"   db        35 seconds ago   Up 34 seconds (healthy)   3306/tcp, 33060/
tcp
foodexpress-web  online-food-ordering-web  "python app.py"          web       34 seconds ago   Up 12 seconds             0.0.0.0:5000->50
00/tcp, [::]:5000->5000/tcp
```

Figure 5.14: Docker Compose services/status

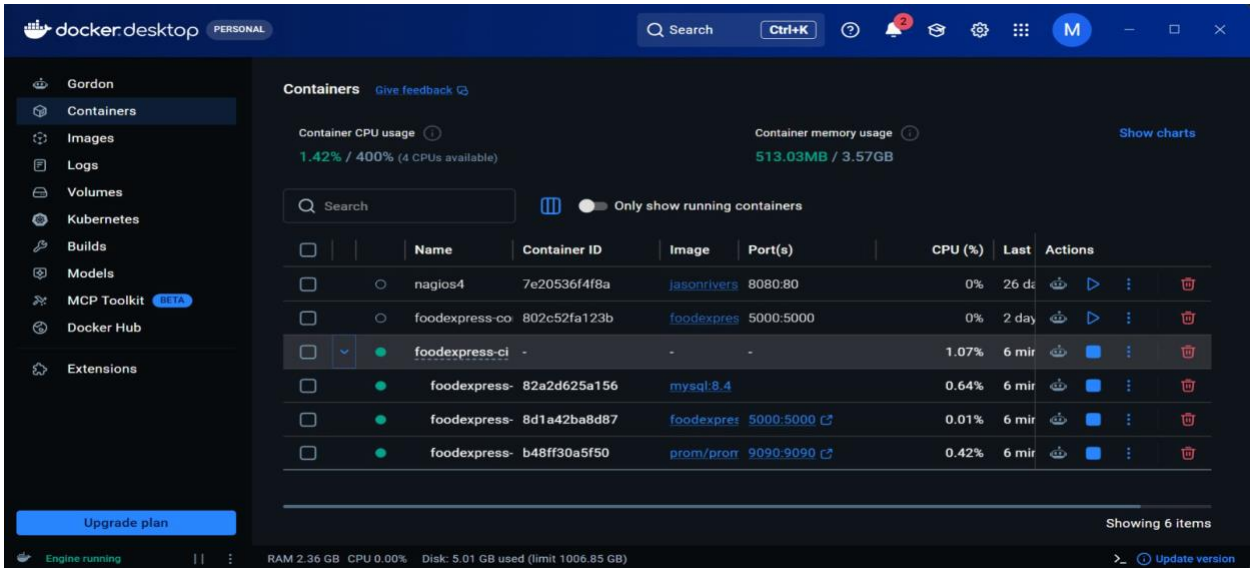


Figure 5.15: Running multi-service environment

5.4 Continuous Integration Using Jenkins

Continuous Integration is a development practice in which developers frequently integrate changes into a shared repository and automated processes build and test those changes. Jenkins is an automation server used for CI/CD activities such as building applications, executing tests and performing deployment operations.

Jenkins was installed and configured on the development machine, and a pipeline named FoodExpress-CI was connected to the FoodExpress GitHub repository. A Jenkinsfile was stored with the project source code. The initial CI pipeline obtained the source code, set up Python dependencies and ran automated tests using pytest.

GitHub and Jenkins were connected using a GitHub webhook. Since Jenkins was running locally, an ngrok tunnel was used to make the Jenkins webhook endpoint temporarily accessible to GitHub. New pushes to GitHub could therefore automatically start the CI pipeline.

Developer
↓
Git Push
↓
GitHub
↓
Webhook
↓
Jenkins
↓
Setup + Automated Tests
↓
SUCCESS / FAILURE

LO3: “To Build and Manage automated CI/CD pipeline, for application development, packaging, testing and deployment.”



Figure 5.16: Jenkins pipeline configuration

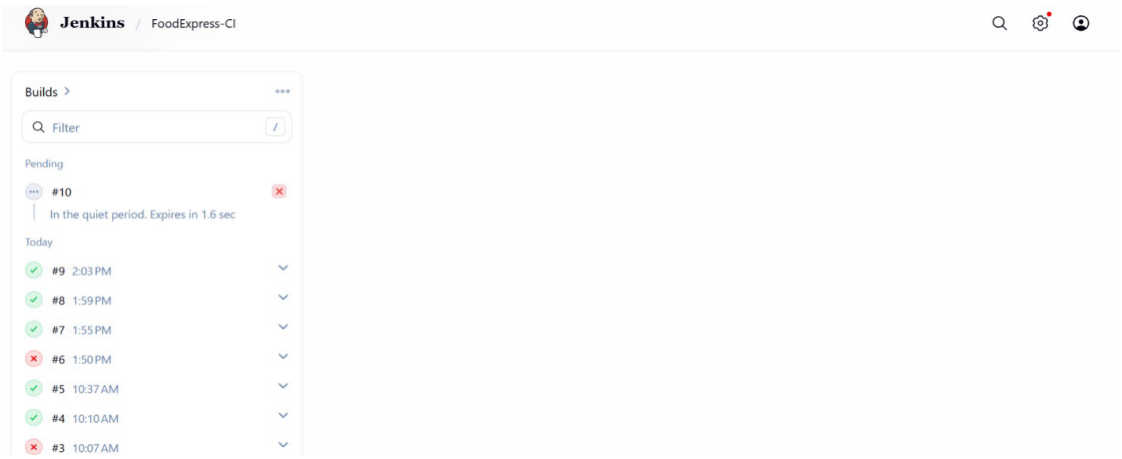


Figure 5.17: Jenkins build execution

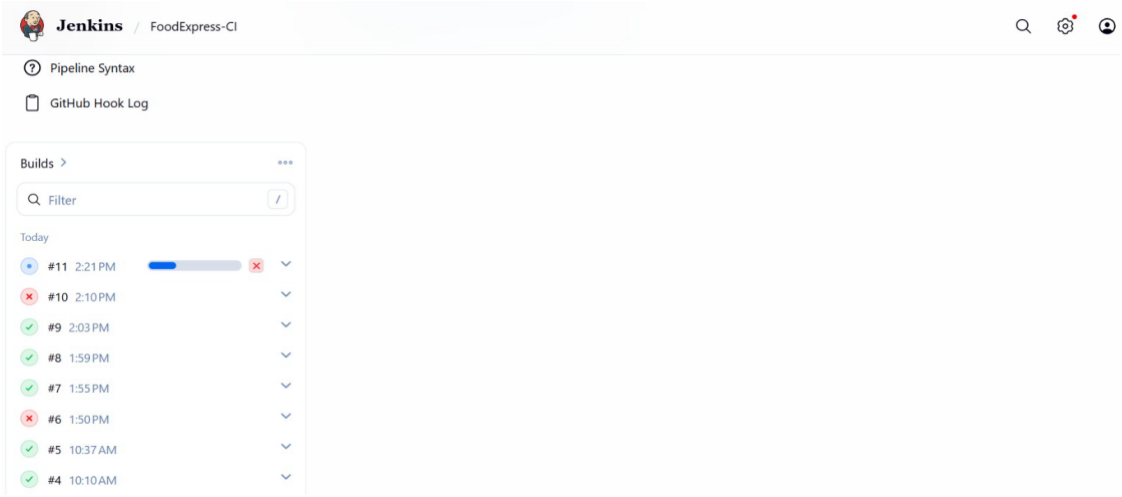


Figure 5.18: Jenkins pipeline result



Figure 5.19: Jenkins build history

5.5 Automated Deployment Using GitHub and Jenkins

It extends Continuous Integration toward Continuous Deployment. The Jenkins pipeline created in Experiment 4 was extended to perform Docker building and deployment.

The resulting workflow became:

Git Push
↓
GitHub
↓
GitHub Webhook
↓
Jenkins
↓
Setup
↓
pytest
↓
Build Docker Image
↓
Docker Compose Deployment
↓
Updated FoodExpress Application

The pipeline first retrieves the latest source code and installs the required dependencies. It then runs the FoodExpress automated tests. Only after the testing stage succeeds does Jenkins proceed to build the latest Docker image and deploy the application using Docker Compose. This makes deployment part of the automated pipeline instead of requiring the developer to repeat the complete sequence manually.

LO3: “Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.”

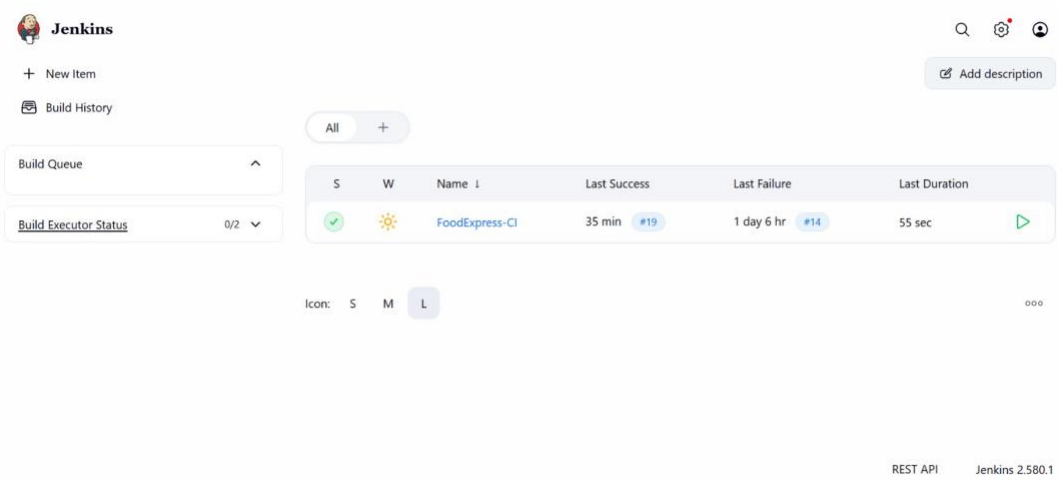


Figure 5.20: Automated deployment / Jenkins pipeline execution

5.6 Agile Software Development Workflow Using Jira

Jira was used as the Agile planning and tracking tool. The experiment connected Agile development, Jira, Git/GitHub, Jenkins, webhooks and Docker-related development activities into one practical workflow.

A Scrum-style workflow was followed:

To Do
↓
In Progress
↓
In Review
↓
Done

The feature task “Add About section to FoodExpress homepage” was created and tracked in Jira. It was moved to In Progress when development began, implemented through Git/GitHub, tested and pushed to the repository. After review and successful Jenkins execution/deployment, the Jira issue was moved to Done.

Jira Task
↓
Development
↓
Git / GitHub
↓
Jenkins Verification
↓
Docker Deployment
↓
Jira Done

LO1: “Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows.”

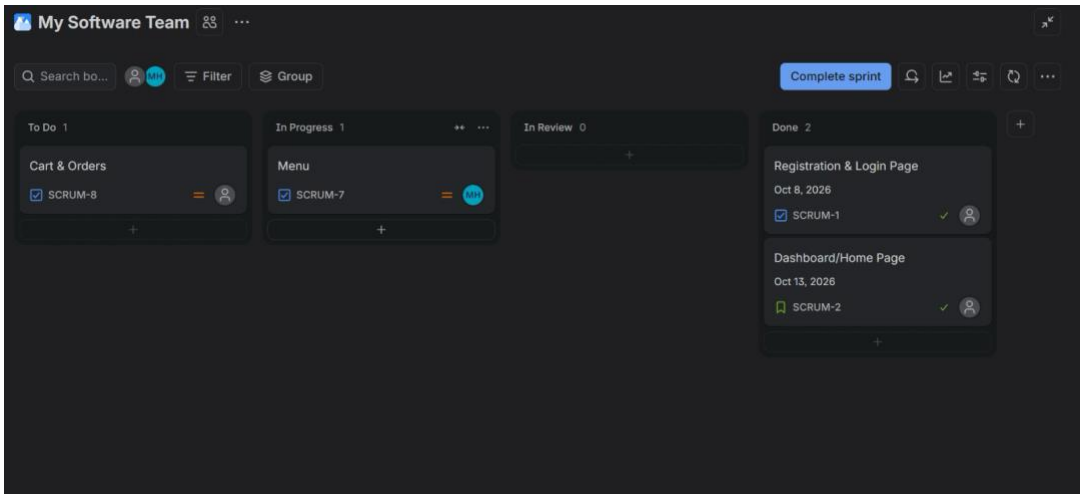


Figure 5.21: Jira project/task tracking

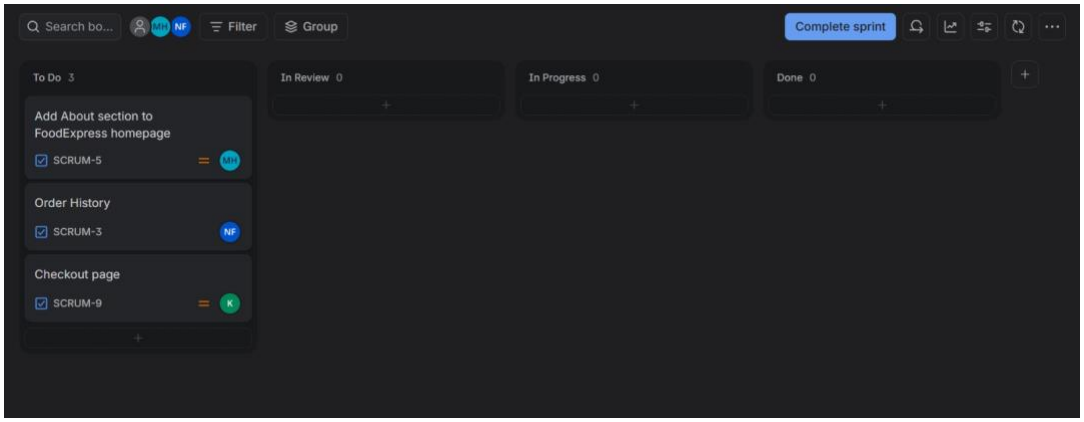


Figure 5.22: Jira issue/workflow evidence

```
Already on 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git pull origin main
From https://github.com/MohitHotwani10/online-food-ordering
 * branch            main       -> FETCH_HEAD
Already up to date.
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git checkout -b SCRUM-5-add-about-section
Switched to a new branch 'SCRUM-5-add-about-section'
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git branch
* SCRUM-5-add-about-section
  main
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git status
On branch SCRUM-5-add-about-section
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app/templates/index.html
```

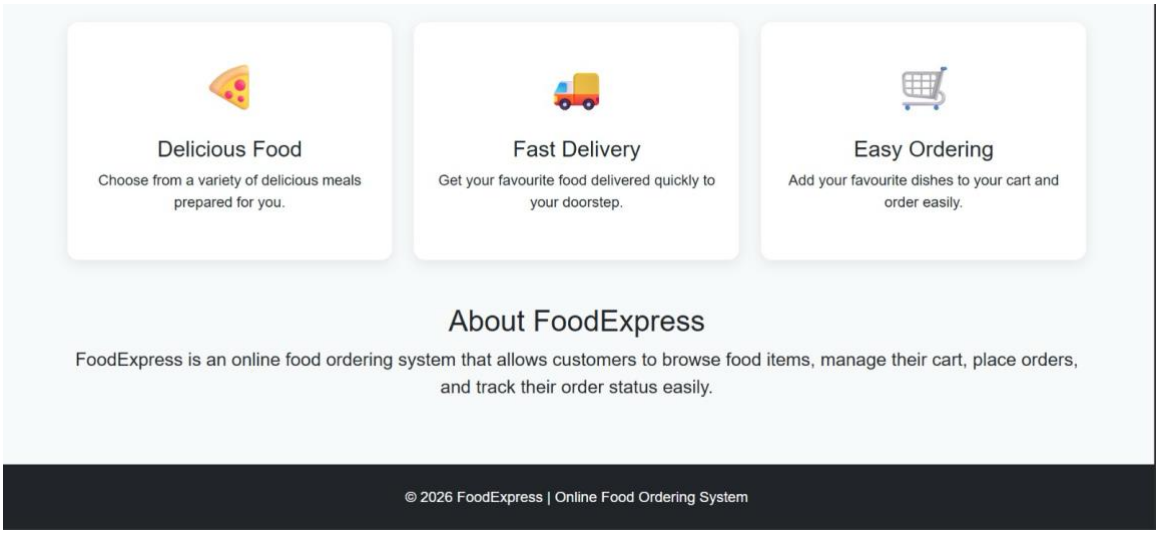


Figure 5.23: FoodExpress About section implementation

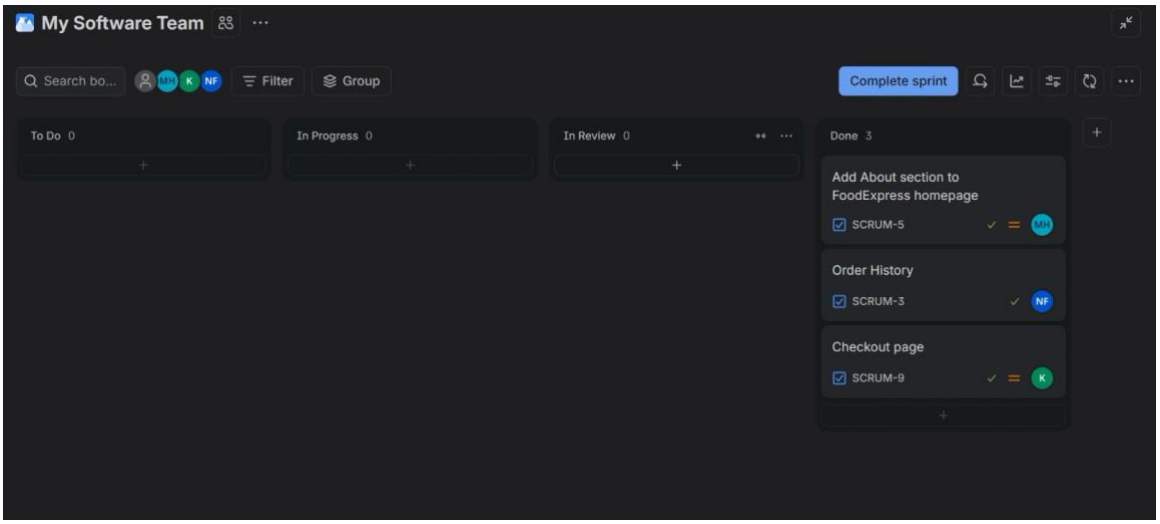


Figure 5.24: Jira software project board

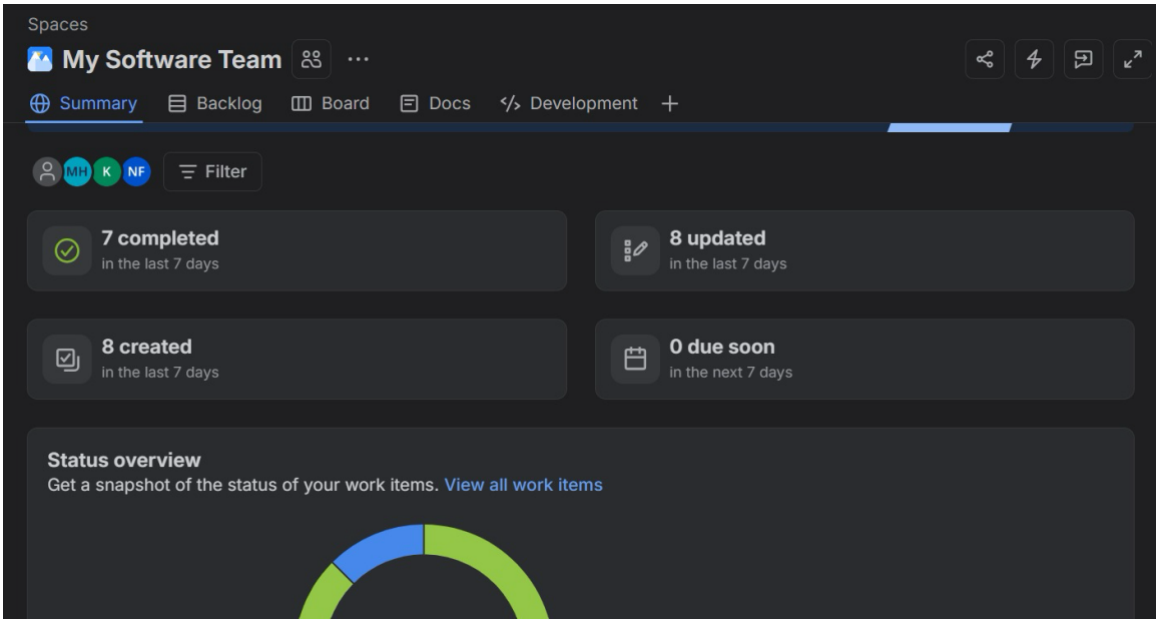


Figure 5.25: Jira project status overview

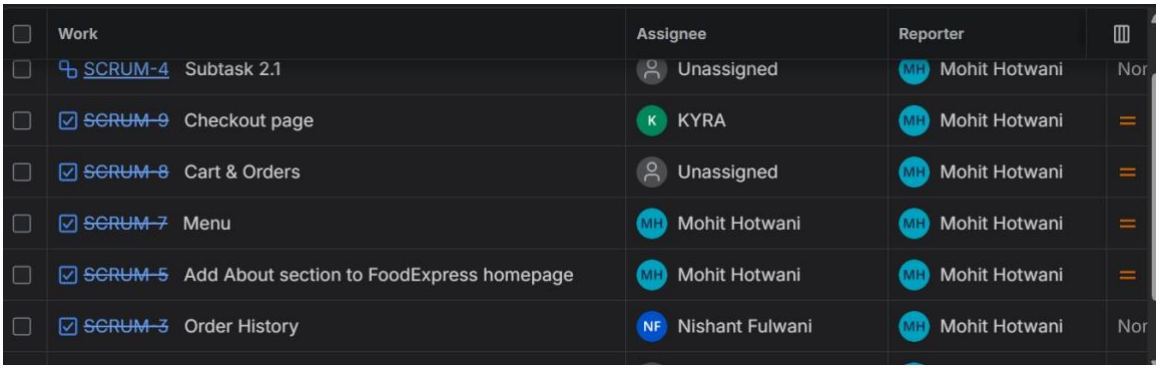


Figure 5.26: Jira issues and team assignments

Figure 5.27: Jira task details

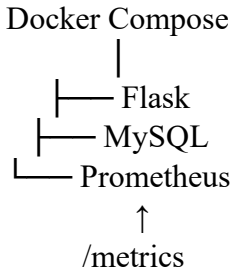
5.7 Continuous Monitoring Using Prometheus

Continuous monitoring involves observing applications, servers, networks and infrastructure so that failures and performance-related problems can be detected after deployment. The experiment covers monitoring through Prometheus, Grafana and Nagios. Prometheus collects and stores time-series metrics, while Grafana can be used to visualize metrics through dashboards.

For FoodExpress, Prometheus was selected as the monitoring technology. The prometheus-flask-exporter package was integrated with the Flask application and exposed application monitoring information through the /metrics endpoint.

A prometheus.yml configuration was created with a FoodExpress scrape job, and Prometheus was added as another service in the Docker Compose configuration. Prometheus accesses the Flask service through web:5000 on the Compose network and periodically scrapes the /metrics endpoint.

The Prometheus target page was used to verify that the FoodExpress application was UP. PromQL queries were used to analyse the application; Flask_http_request_total was used to examine HTTP requests according to method and response status, while request-duration metrics allowed monitoring beyond simply checking whether the container was running.



LO5: “Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.”

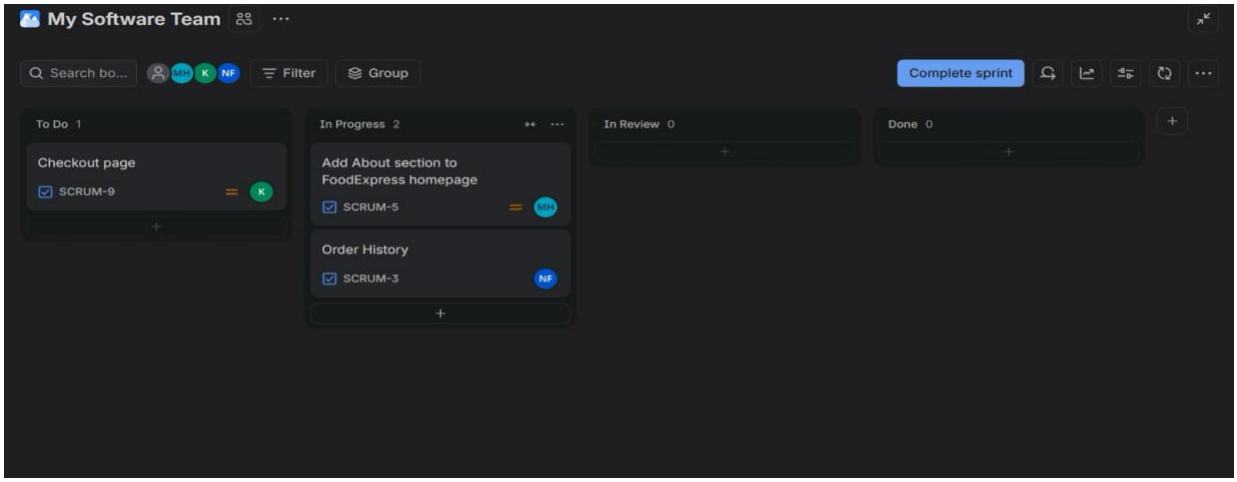


Figure 5.28: Prometheus configuration / target evidence

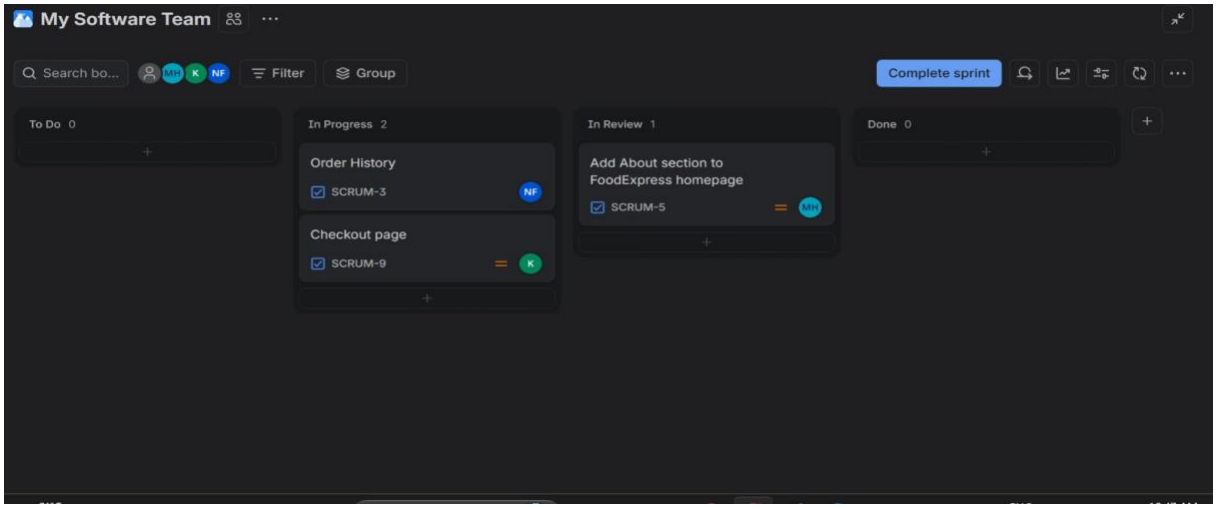


Figure 5.29: Prometheus monitoring interface

```
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git add app/templates/index.html
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git commit -m "SCRUM-5 Add About section to homepage"
[SCRUM-5-add-about-section 19a00b8] SCRUM-5 Add About section to homepage
 1 file changed, 10 insertions(+), 1 deletion(-)
PS C:\Users\kunal\OneDrive\Desktop\online-food-ordering> git push -u origin SCRUM-5-add-about-section
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 667 bytes | 667.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
remote:
remote: Create a pull request for 'SCRUM-5-add-about-section' on GitHub by visiting:
remote:      https://github.com/MohitHotwani10/online-food-ordering/pull/new/SCRUM-5-add-about-section
remote:
To https://github.com/MohitHotwani10/online-food-ordering.git
```

Figure 5.30: Monitoring command/configuration

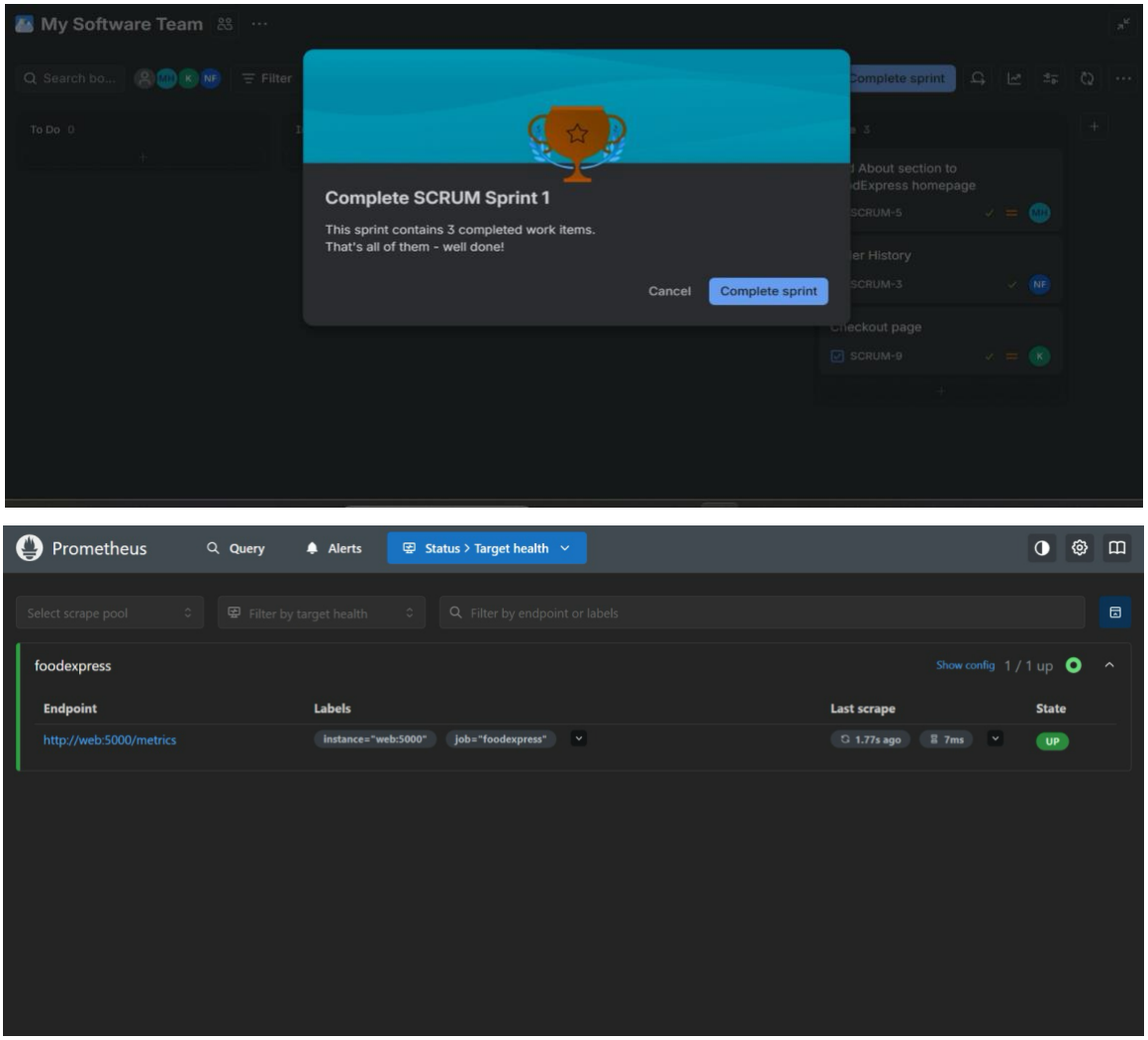


Figure 5.31: Prometheus target status showing FoodExpress UP

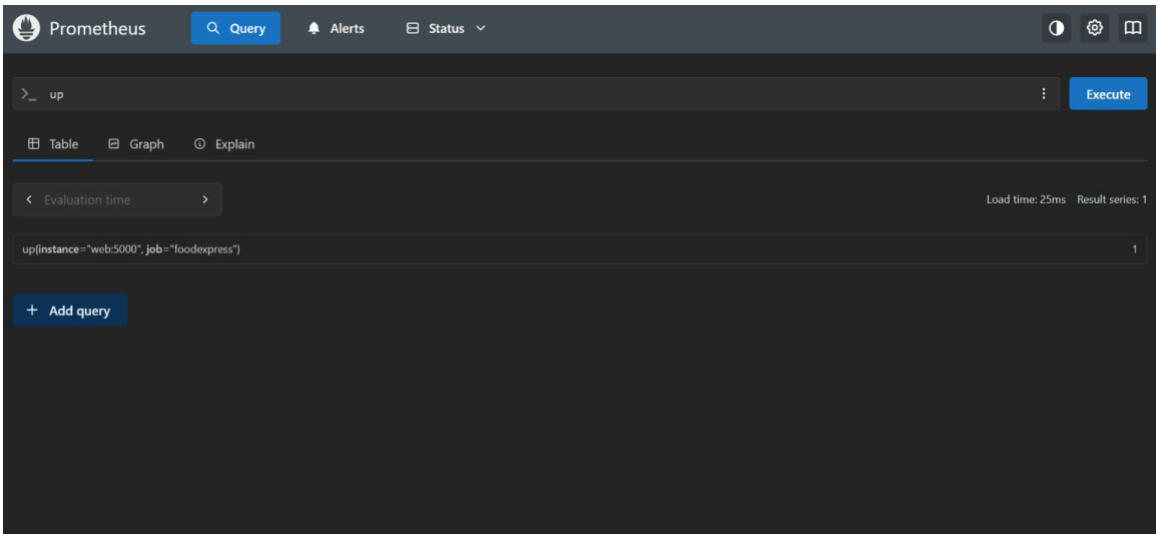


Figure 5.32: Prometheus query interface

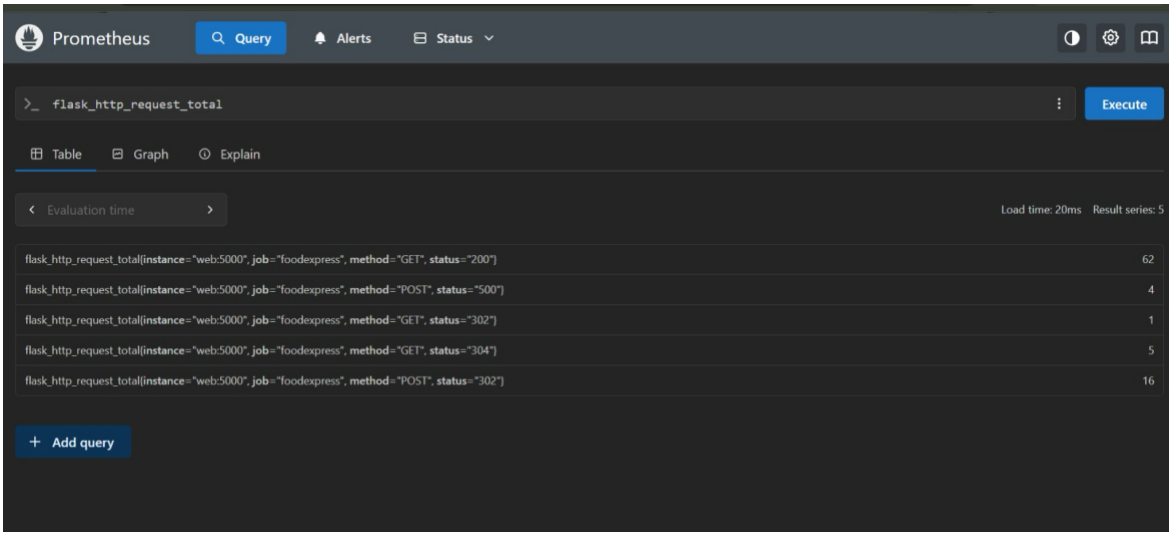


Figure 5.33: Prometheus metric query results

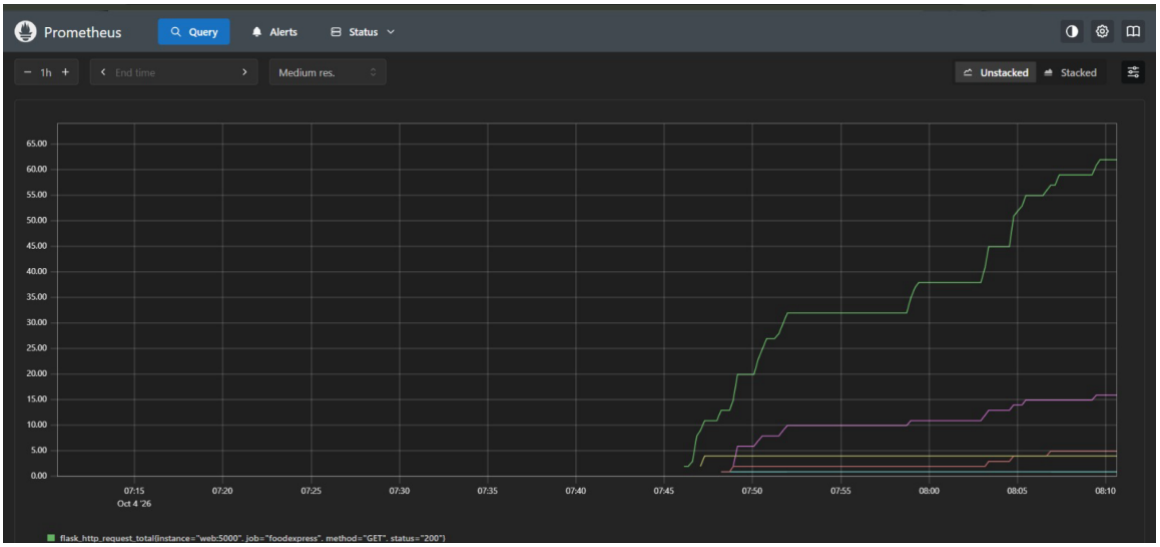


Figure 5.34: Prometheus time-series metric visualization

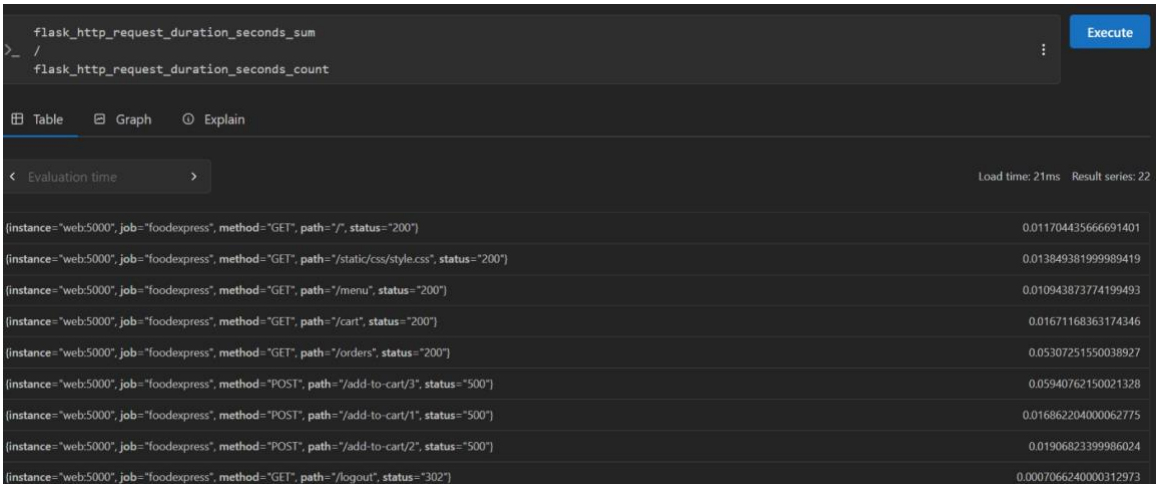


Figure 5.35: Prometheus HTTP request metrics

Figure 5.36: Prometheus metric visualization

6. Testing

6.1 Automated Testing

Testing was incorporated into the Jenkins Continuous Integration pipeline. The Jenkinsfile included the setup of Python dependencies and execution of automated tests using pytest. The tests were therefore executed as part of the CI workflow rather than only as a manual local activity.

6.2 CI Verification

The CI workflow was verified through the sequence Developer → Git Push → GitHub → Webhook → Jenkins → Setup + Automated Tests → SUCCESS / FAILURE. The deployment extension in Experiment 5 additionally required the testing stage to succeed before Docker image building and Docker Compose deployment proceeded.

6.3 Monitoring Verification

Monitoring verification was performed by checking the Prometheus target page and confirming that the FoodExpress application was UP. PromQL was then used to inspect HTTP request and request-duration metrics. This verified that monitoring was integrated with the deployed application.

6.4 Testing Evidence

The source report contains screenshots of GitHub, Docker, Docker Compose, Jenkins, Jira and Prometheus activities. These screenshots provide practical evidence of the implementation and verification stages described in the experiments.

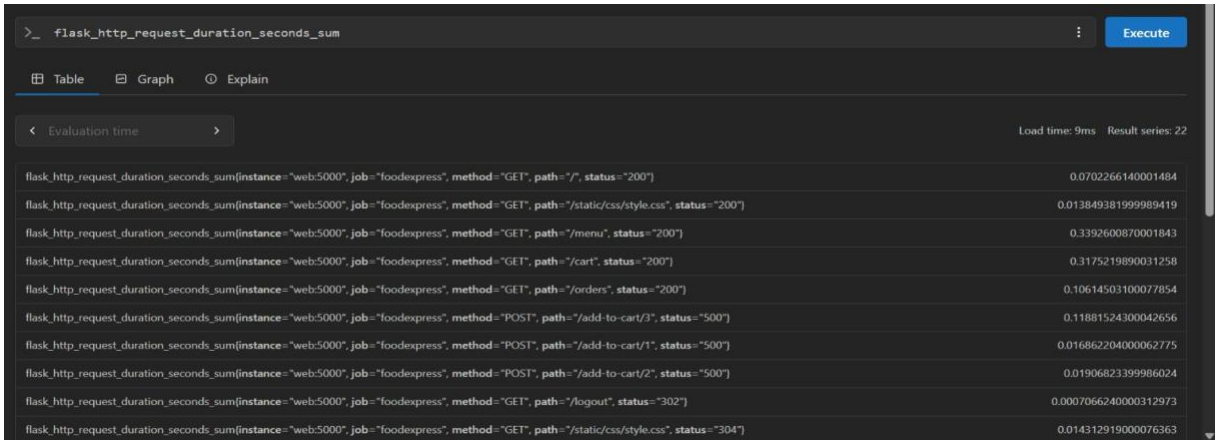
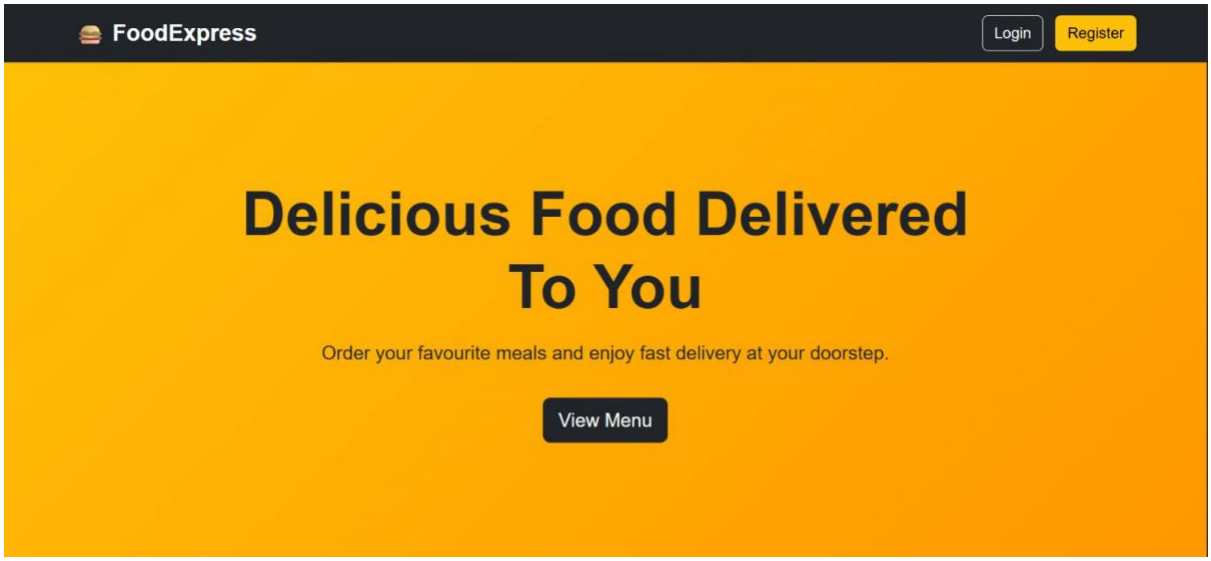
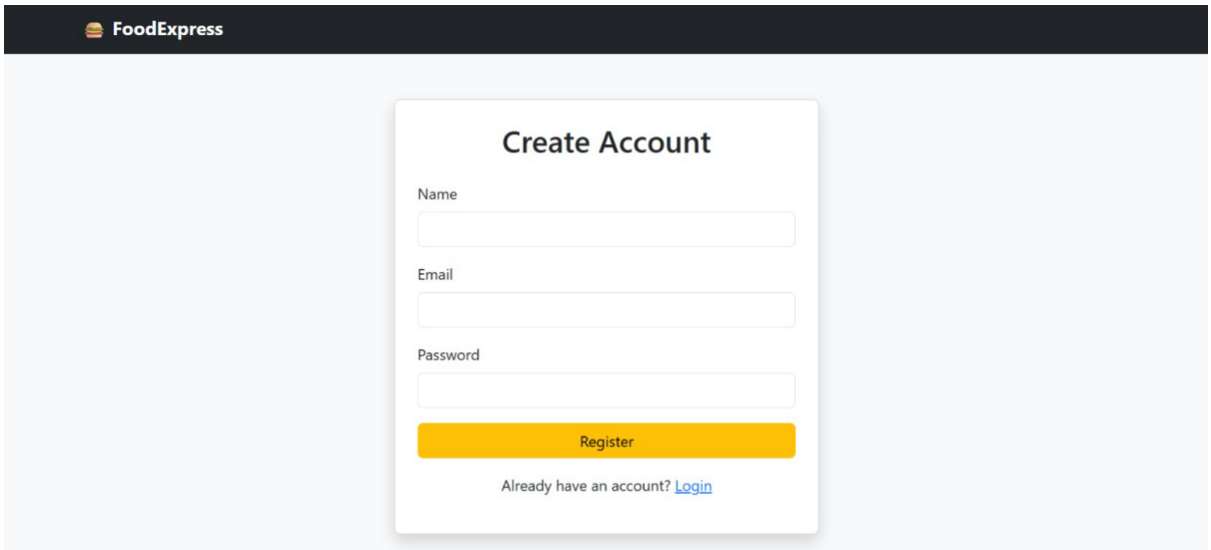
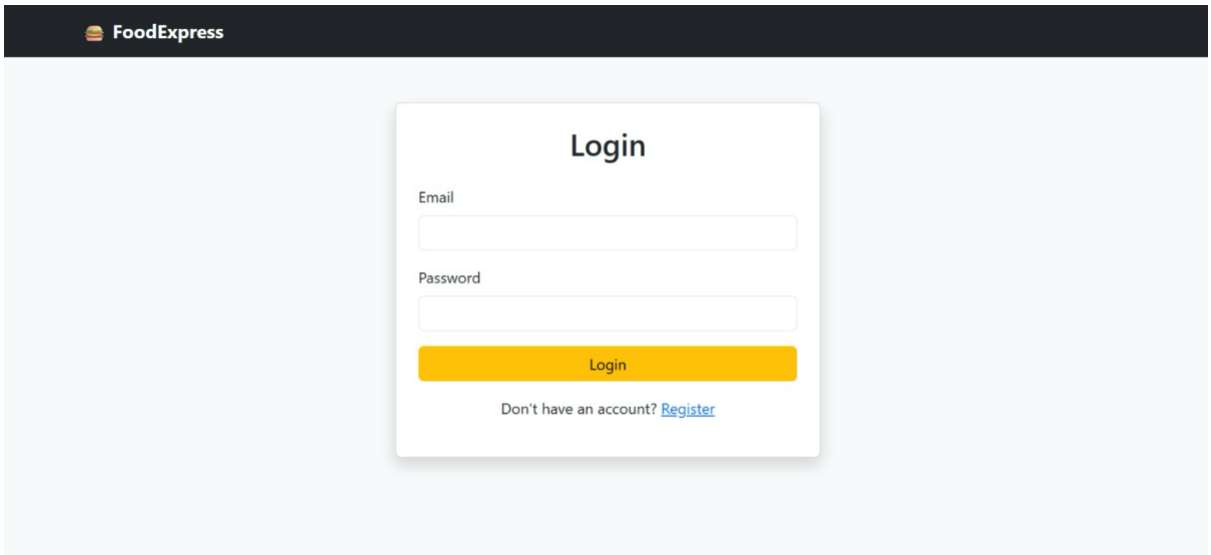


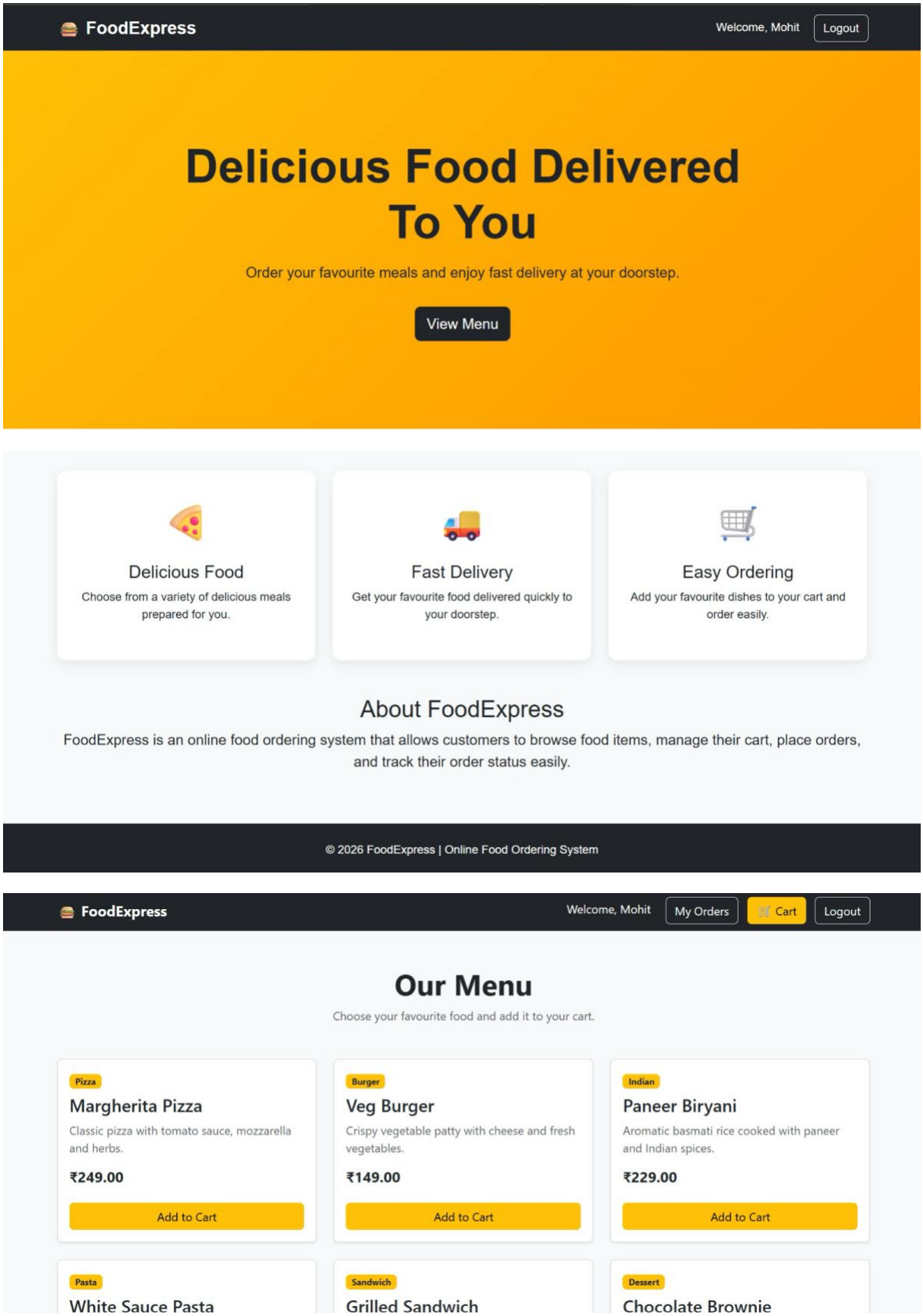
Figure 6.1: Automated testing / CI evidence

7. Implementation:

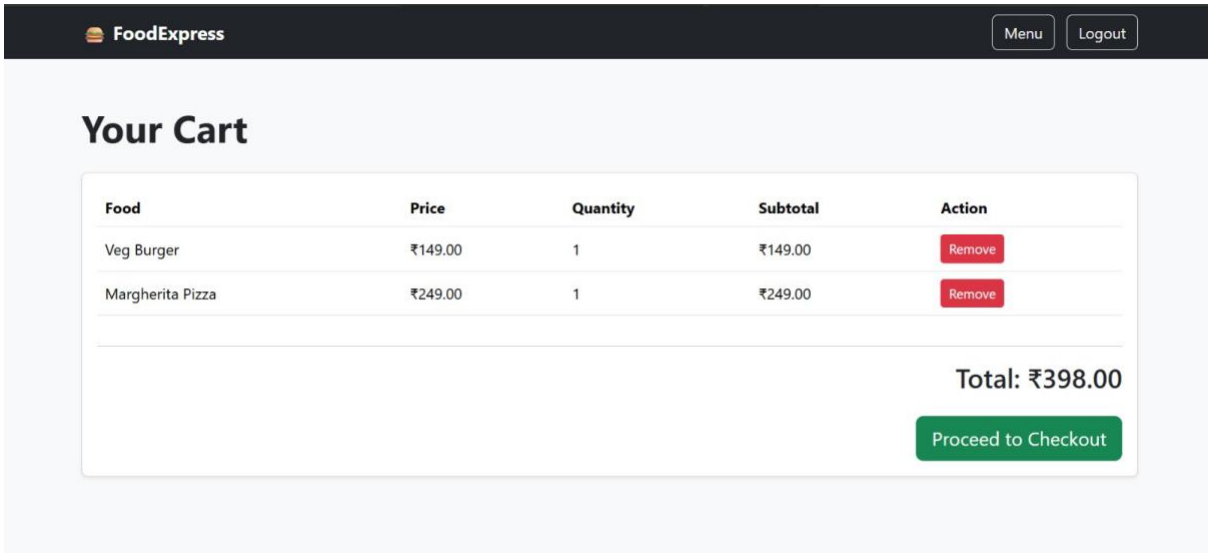


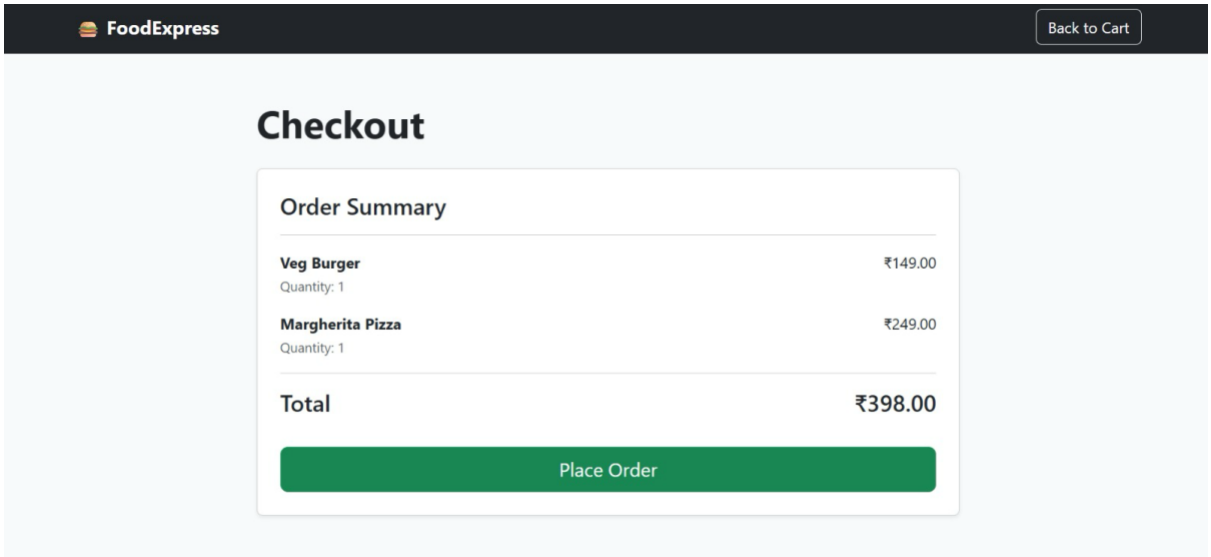


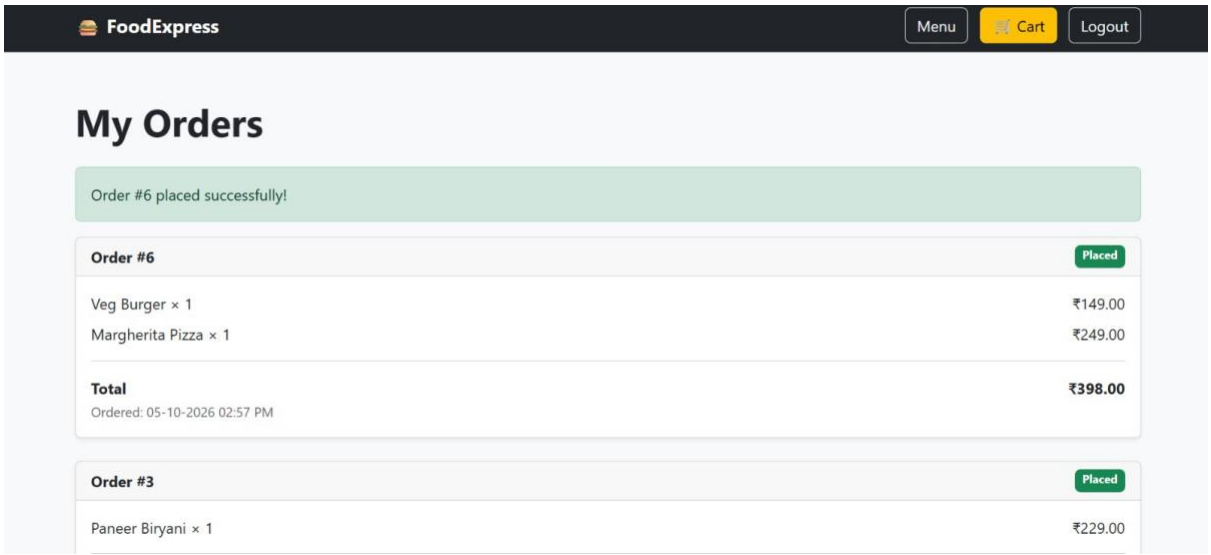


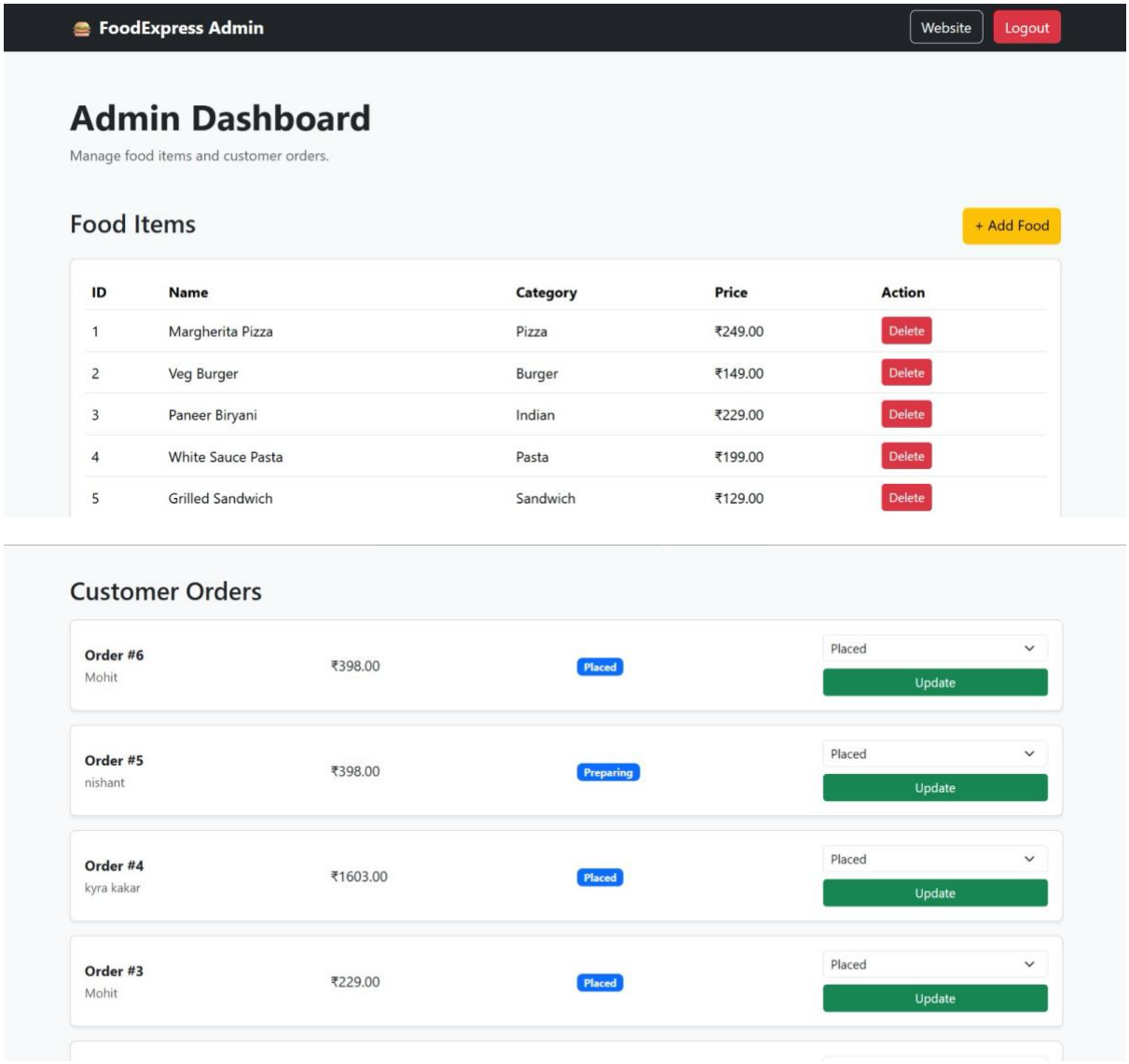

FoodExpress
Welcome, Mohit
Logout
Delicious Food Delivered To You
Order your favourite meals and enjoy fast delivery at your doorstep.
View Menu
Delicious Food
Choose from a variety of delicious meals prepared for you.
Fast Delivery
Get your favourite food delivered quickly to your doorstep.
Easy Ordering
Add your favourite dishes to your cart and order easily.
About FoodExpress
FoodExpress is an online food ordering system that allows customers to browse food items, manage their cart, place orders, and track their order status easily.
© 2026 FoodExpress | Online Food Ordering System

FoodExpress
Welcome, Mohit
My Orders
Cart
Logout
Our Menu
Choose your favourite food and add it to your cart.
Pizza
Margherita Pizza
Classic pizza with tomato sauce, mozzarella and herbs.
₹249.00
Add to Cart
Burger
Veg Burger
Crispy vegetable patty with cheese and fresh vegetables.
₹149.00
Add to Cart
Indian
Paneer Biryani
Aromatic basmati rice cooked with paneer and Indian spices.
₹229.00
Add to Cart
Pasta
White Sauce Pasta
Sandwich
Grilled Sandwich
Dessert
Chocolate Brownie


FoodExpress
Menu
Logout
Your Cart
Food
Price
Quantity
Subtotal
Action
Veg Burger
₹149.00
1
₹149.00
Remove
Margherita Pizza
₹249.00
1
₹249.00
Remove
Total: ₹398.00
Proceed to Checkout


FoodExpress
Back to Cart
Checkout
Order Summary
Veg Burger
Quantity: 1
₹149.00
Margherita Pizza
Quantity: 1
₹249.00
Total
₹398.00
Place Order


FoodExpress
Menu
Cart
Logout
My Orders
Order #6 placed successfully!
Order #6
Placed
Veg Burger × 1
₹149.00
Margherita Pizza × 1
₹249.00
Total
₹398.00
Ordered: 05-10-2026 02:57 PM
Order #3
Placed
Paneer Biryani × 1
₹229.00



8. Results and Discussion

8.1 Results

Area	Result
Version control	FoodExpress source code was maintained in Git and pushed to GitHub; branches and merges were used for feature development.
Containerization	The Flask application was built into a Docker image and executed as a container with port 5000 mapped to the host.
Multi-service deployment	Docker Compose ran the Flask application and MySQL as interconnected services with persistent database storage and health checking.
Continuous Integration	Jenkins was connected to GitHub through a webhook and executed setup and pytest stages automatically after changes.

Automated deployment	The Jenkins pipeline was extended to build the Docker image and deploy the application using Docker Compose after successful testing.
Agile workflow	Jira was used to create and track a practical feature through To Do, In Progress, In Review and Done.
Continuous monitoring	Prometheus collected Flask application metrics through /metrics and supported PromQL-based analysis.

Table 7.1: Summary of project results

8.2 Discussion

The project demonstrates that the individual laboratory experiments can be combined into one software-development lifecycle. Git and GitHub provide the source-control foundation; Docker and Docker Compose provide a consistent and multi-service execution environment; Jenkins automates verification and delivery; Jira connects development work with Agile planning; and Prometheus extends the lifecycle into continuous monitoring.

The integrated workflow reduces repetitive manual operations and provides a clear relationship between source-code changes, testing, deployment, Agile task tracking and monitoring. The source report specifically concludes that the project transformed the individual laboratory concepts into a connected practical DevOps workflow.

9. Conclusion and Future Scope

9.1 Conclusion

The development of FoodExpress – Online Food Ordering System provided practical experience in applying software-development and DevOps concepts to a single integrated application.

Git and GitHub provided structured version control, commit history, branching and remote source-code management. Docker packaged the application and its dependencies into a portable containerized environment, while Docker Compose coordinated Flask and MySQL as interconnected services with networking and persistent database storage.

Jenkins demonstrated how Continuous Integration can automatically verify changes through testing and how a CI/CD pipeline can further automate Docker image building and application deployment. Connecting GitHub and Jenkins through a webhook showed how a source-code change can automatically initiate the delivery pipeline, reducing repetitive manual operations and improving consistency.

Jira introduced the practical application of Agile and Scrum principles by representing software features as manageable tasks and moving them through stages of development, review and completion.

Prometheus demonstrated that the DevOps lifecycle does not end after deployment. By exposing Flask metrics and querying them through Prometheus, application traffic, HTTP status codes, request counts and request-duration information can be observed to understand the behaviour and health of a running system.

Overall, the FoodExpress mini project helped demonstrate how version control, application development, containerization, multi-service deployment, CI/CD automation, Agile project management and continuous monitoring work together as parts of a complete software-development lifecycle.

9.2 Future Scope

The supplied FoodExpress report does not specify a separate future-scope section. Based strictly on the technologies and workflow already demonstrated, the project can be extended by adding richer Agile planning, further automated tests, additional deployment environments and broader monitoring/visualization. These are potential extensions rather than features claimed as implemented in the source report.

9.3 Learning Outcome

The project provided hands-on experience in developing, deploying, automating, managing and monitoring a web application while connecting the individual ASD & DevOps laboratory experiments into one practical workflow.

A. Appendix: Running the Project

A.1 Docker Build

docker build -t foodexpress .

This command builds the FoodExpress Docker image from the project Dockerfile.

A.2 Docker Container

The FoodExpress container maps port 5000 of the container to port 5000 of the host. Commands such as docker images, docker ps, docker logs, docker stop and docker start were used during the implementation.

A.3 Docker Compose

docker compose config
docker compose up -d
docker compose ps
docker compose logs

These commands were used for Compose configuration validation, startup, service-status checking and troubleshooting.

A.4 Git

git log --oneline
git remote add origin <repository>
git push -u origin main

Git commands were used to inspect commit history, connect the local repository to GitHub and push changes.

A.5 Monitoring

/metrics

The Flask application exposed monitoring data through the /metrics endpoint, which Prometheus periodically scraped for analysis.